

Database Models

A **database model** defines the logical structure of a database and determines how data is stored, organized, and manipulated. There are several key database models:

1. Hierarchical Model:

- Organizes data into a tree-like structure with parent-child relationships.
- Each parent can have multiple children, but each child has only one parent.
- Example: Organizational structures, file systems.

2. Network Model:

- More flexible than the hierarchical model; allows many-to-many relationships.
- Data is represented as records connected by links.
- Example: Telecommunications networks, transport systems.

3. Relational Model:

- Most widely used model, organizes data into tables (relations) of rows and columns.
- Tables are connected through keys (Primary, Foreign keys).
- Uses Structured Query Language (SQL) for data manipulation.
- Example: Customer database, order processing systems.

4. Object-Oriented Model:

- Stores data in objects, similar to object-oriented programming.
- Supports inheritance, polymorphism, and encapsulation.
- Useful for complex data structures like multimedia, CAD systems.

5. Document-Based Model (Non-relational):

- Stores data as documents, typically in JSON, BSON, or XML format.
- Commonly used in non-relational databases (NoSQL), ideal for unstructured or semi-structured data.
- Example: MongoDB, CouchDB.

6. Key-Value Model (Non-relational):

- Simple model where data is stored as key-value pairs.
- Fast access and high scalability, used for caching or simple lookups.
- Example: Redis, DynamoDB.

Differences Between Relational and Non-Relational Database Models

Feature	Relational Databases	Non-Relational Databases
Data Structure	Tables (Rows & Columns)	Documents, Key-Value Pairs, Graphs, Columns
Schema	Fixed schema (requires predefined structure)	Flexible schema (structure can evolve)
Normalization	Data is normalized (minimizes redundancy)	Denormalization is common (for speed and simplicity)
Scalability	Vertical scaling (adding more resources to a single server)	Horizontal scaling (distributes data across multiple servers)
Query Language	SQL (Structured Query Language)	NoSQL (or query language specific to database)
ACID Compliance	Supports ACID (Atomicity, Consistency, Isolation, Durability)	May or may not support ACID (depends on type)
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, DynamoDB, Couchbase

Key Considerations:

- Relational Databases are better for structured data with complex relationships, where consistency and transactions are critical.
- Non-relational Databases are ideal for unstructured or semi-structured data, high-performance applications, and systems requiring large scalability.

Modeling a Relational Database

Step 1: Define Entities and Attributes

- Identify key objects in the system (entities). For example, in a library system, entities could be "Books," "Authors," "Members."
- List attributes (fields) for each entity. For example, for "Books," attributes could be Title, Author, Publication Year, ISBN.

Step 2: Define Primary Keys

- Select a unique identifier for each entity.
- Example: In "Books," the primary key could be ISBN.

Step 3: Define Relationships

- Identify how entities are related. Relationships can be:
 - One-to-one: Each entity in a relationship is linked to one other entity.
 - One-to-many: One entity can be linked to many entities.
 - Many-to-many: Multiple entities are linked to many other entities.
- Example: A "Member" can borrow many "Books" (one-to-many).

Step 4: Create Foreign Keys

- Establish foreign keys to enforce relationships between tables.
- Example: "Member ID" from the "Members" table could be a foreign key in the "Borrowed Books" table.

Step 5: Normalize the Database

- Ensure that the database is normalized by eliminating redundancy and ensuring efficient data storage.
- Apply normalization forms (1NF, 2NF, 3NF) to avoid data anomalies.

Example Database Schema for a Library System:

- Books Table:

- ISBN (Primary Key)
- Title
- Author
- Publication Year

- Members Table:

- Member ID (Primary Key)
- Name
- Email
- Phone Number

- Borrowed Books Table:

- Borrow ID (Primary Key)
- Member ID (Foreign Key)
- ISBN (Foreign Key)
- Borrow Date
- Return Date

Choosing a Database Model Given a Scenario

When selecting a database model, consider the following factors:

- **Data Structure:** Is the data structured, semi-structured, or unstructured?
 - If structured: Relational Model (e.g., SQL).
 - If unstructured or semi-structured: Non-Relational Model (e.g., NoSQL).
- **Volume of Data:** How much data will the database manage?
 - If you expect large-scale data with high throughput needs: Non-Relational Model (for scalability).
 - For smaller, well-structured datasets: Relational Model.
- **Complexity of Relationships:** Does the application need to handle complex relationships or transactions?
 - For complex relationships: Relational Model.
 - For simpler, document-based data: Non-Relational (Document or Key-Value Model).
- **Performance & Scalability:** Is performance a critical factor, and does the application require high scalability?
 - If yes: Consider Non-Relational Models for their horizontal scaling abilities (e.g., distributed databases).
 - If ACID compliance and consistency are critical: Relational Model (e.g., MySQL, PostgreSQL).

Examples:

- E-commerce Platform: Requires managing product inventories, orders, customer accounts (Relational Model).
- Social Media Application: Handles unstructured user-generated content, multimedia, and requires scalability (Non-Relational Model).

Exercise:

- Given a blog application that stores posts, comments, and user profiles, decide whether a relational or non-relational database is suitable and explain why.